

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

Laboratorio 3: Secured Remote Exe

Laboratorio 3 de Fundamentos de Seguridad

Semestre 2026-2

Integrantes:

Tomás Olaya Díaz
Juan Pablo Vega Villamil

Profesor:

Fabian Eduardo Sierra Sanchez

Bogotá D.C

Fecha de entrega: 10 de Septiembre de 2026

Informe de Laboratorio:

Nos organismos de la siguiente manera

Tomás Olaya Díaz, Red Team con Kali Linux, y Juan Pablo Vega, Blue Team con Arch Linux.

Introducción

Para este laboratorio construimos un escenario donde un backend en Spring Boot expone un inventario de dispositivos de red ficticio y un simulador de scripts de administración, pensado como una versión simplificada de las herramientas que un equipo de redes usaría para no tener que revisar dispositivo por dispositivo de forma manual. La idea era tener un servicio real, aunque sea de juguete, contra el cual pudiéramos ejercer los dos roles del reto: yo atacué el sistema desde Kali Linux mientras Juan Pablo lo administraba y monitorear desde una máquina con Arch Linux. El objetivo no era romper nada de forma gratuita, sino levantar evidencia concreta de qué tan expuesta queda una aplicación cuando se despliega rápido y sin las protecciones básicas de un entorno productivo.

Arquitectura desplegada

El despliegue quedó en una sola VM local con la IP 10.212.92.54. Ahí corre el backend en Spring Boot, un frontend en React compilado como aplicación de una sola página y Nginx como reverse proxy en el puerto 80, encargado de servir los archivos estáticos del frontend y de reenviar las peticiones de la API hacia el backend. Como es un laboratorio y no un entorno de producción, dejamos el servicio sin autenticación y sin HTTPS a propósito, ya que ese es justamente el punto de partida que después hay que corregir. Esa decisión de diseño es la que terminó explicando buena parte de los hallazgos que describimos más adelante.

Reconocimiento, Red Team

Antes de arrancar las pruebas definir las variables de entorno con la IP objetivo y dejé registrada la hora de inicio con `date -u`, que marcó 2026-09-09T14:42:52Z, para poder ubicar después cada hallazgo en el tiempo.

Lo primero que corrí fue un escaneo con Nmap contra el puerto 80, con el comando `nmap -Pn -sV -p 80 $TARGET_IP -oA evidence/red/nmap_port80`. El resultado confirmó el servicio activo con 80/tcp open http nginx 1.30.4, es decir que Nmap no solo detectó el puerto abierto sino que logró identificar la versión exacta del servidor web sin necesitar ningún esfuerzo adicional. Ese mismo dato de versión reapareció después como hallazgo pasivo en ZAP, así que quedó confirmado por dos herramientas distintas.

Con curl probé primero la raíz del sitio con `curl -i $TARGET_URL/`, y la respuesta llegó en texto plano sin cifrar: encabezados `Server nginx/1.30.4`, `Content-Type text/html`, `Content-Length 350` y un `ETag`, seguidos del HTML completo del index de la aplicación, que carga el bundle `/assets/index-CfnRBC6C.js`. Repetí la prueba contra `$TARGET_URL/public-inventory.txt` esperando encontrar un archivo con datos de inventario, pero la respuesta fue prácticamente idéntica a la anterior, mismo `Content-Length 350` y el mismo HTML del shell de la aplicación en lugar de un listado de dispositivos. Lo más probable es que esa ruta no exista como archivo estático real y que Nginx esté cayendo al `try_files` hacia `index.html`, típico de un despliegue de SPA donde cualquier ruta no reconocida termina sirviendo la misma página. Vale la pena aclarar que en ese mismo comando cometí un error de sintaxis, usé el símbolo de exclamación en vez de la tubería antes de `tee`, lo que disparó unos errores de bash después de que la respuesta ya se había impreso. No afectó la evidencia capturada, pero se nota en la captura.

En Wireshark dejamos corriendo la captura mientras interactuábamos con la aplicación desde el navegador. Se ve con claridad una petición `GET /` seguida de un `304 Not Modified`, y después una petición `GET /api/devices` que en ese momento respondió `502 Bad Gateway`, coincidiendo con el error que también vimos en pantalla al abrir la sección de Inventario. Lo más relevante de la captura fue interceptar la petición que se dispara al usar el Simulador de scripts. Al llenar el formulario con Device ID 1 y un mensaje de prueba y darle a Simular, Wireshark capturó el paquete completo del `POST` a `/api/scripts/simulate`, con el JSON del body totalmente legible en texto plano:

```
{“deviceId”:”1”,”scriptName”:”Hola profe Fabian somos el grupo  
OlayaVega”,”parameters”:”Ya logramos ver los datos requeridos “}
```

Esa captura se hizo en la misma ventana en que el backend todavía no estaba arriba, por lo que la respuesta que recibimos fue un `502 Bad Gateway`. Aun así, la petición sí alcanzó a viajar completa desde el navegador hasta Nginx antes de fallar en el proxy, así que el hallazgo se mantiene: cualquiera que esté en la misma red y pueda capturar tráfico ve exactamente qué dispositivo se está intentando modificar y qué parámetros se están enviando, sin que haga falta descifrar nada.

Nota pendiente: falta agregar aquí la captura posterior con respuesta `200 OK`, tomada una vez el backend ya estaba desplegado, para mostrar que el mismo problema se mantiene incluso cuando la petición sí se procesa correctamente.

Por último corrimos un análisis pasivo con OWASP ZAP usando Manual Explore contra <http://10.212.92.54>. Los hallazgos reales que arrojó fueron los siguientes:

- Content Security Policy Header Not Set, riesgo medio, confianza alta, visto en la raíz, en /api/devices, en /api/scripts/simulate/history y en /public-inventory.txt, CWE-693.
- - Missing Anti-clickjacking Header, riesgo medio, por ausencia de X-Frame-Options o de una directiva frame-ancestors en la CSP, CWE-1021.
- - Server Leaks Version Information via Server HTTP Response Header Field, riesgo bajo, confianza alta, con nginx/1.30.4 como evidencia directa, CWE-200. Es el mismo dato que ya habíamos visto con Nmap.
- - X-Content-Type-Options Header Missing, riesgo bajo.
- - Information Disclosure, Suspicious Comments, informativo, sobre el archivo /assets/index-CfnRBC6C.js. ZAP marcó tres coincidencias del patrón SELECT dentro del bundle minificado. Es un hallazgo que hay que tomar con cuidado porque en JavaScript compilado y minificado es común que aparezcan fragmentos de texto que calzan con ese patrón sin que sean sentencias SQL reales, así que lo dejamos anotado como una alerta a revisar manualmente y no como una confirmación de que el bundle expone consultas.
- - Modern Web Application, informativo. ZAP detectó el uso de un script tipo módulo y sugiere que Ajax Spider sería más efectivo que el spider tradicional para explorar la app a fondo.

Detección, Blue Team

Mientras yo hacía el reconocimiento, Juan Pablo estaba revisando en paralelo los logs directamente en el servidor, cuyo hostname es deltax y corre Arch Linux. Con `sudo journalctl -u nginx --since '20 minutes ago' --no-pager` encontró una serie de errores `connect() failed`, `Connection refused`, apuntando al upstream <http://127.0.0.1:8081>, asociados a peticiones como GET /api/devices y POST /api/scripts/simulate que llegaban desde 10.212.92.142. Esos logs coinciden en tiempo con los 502 que yo estaba viendo tanto en el navegador como en Wireshark, y confirman que la causa era clara y quedaba perfectamente diagnosticable desde el lado del servidor: el backend en el puerto 8081 todavía no se había levantado cuando empezamos las pruebas contra el frontend. En un momento incluso aparece un error de `no live upstreams while connecting to upstream` para la petición del simulador, que es la misma petición que yo había capturado en Wireshark.

Por otro lado, Juan Pablo intentó contar los 404 del access.log con un comando en awk sobre ese archivo, pero en la evidencia que tenemos no quedó capturado el resultado de ese comando, así que no podemos afirmar cuántos 404 se registraron ni cuáles rutas los generaron. Lo que sí filtró con éxito fue el tráfico de las herramientas

de reconocimiento usando grep sobre el access.log, buscando nmap, curl y ZAP. Ahí se ve con el user agent de Nmap Scripting Engine peticiones a rutas como NmapLowerCheck, NmapUpperCheck y Nmap/folder/check, además de un POST a /sdk que respondió 405. Esas rutas no las pedimos nosotros de forma manual, las genera automáticamente la detección de versión de Nmap al sondear el servicio. El log muestra ese patrón repetido en dos momentos distintos, uno cerca de las 09:49 y otro cerca de las 10:10 hora local, lo que indica que el escaneo de Nmap se corrió dos veces durante la sesión. También quedaron registradas ahí las peticiones directas con curl, tanto la de la raíz como la de public-inventory.txt.

Una vez el backend quedó arriba, el problema desapareció: al volver a cargar el Inventario de dispositivos, la aplicación respondió con normalidad mostrando los cinco dispositivos ficticios registrados, dos firewalls, un router y dos switches, cada uno con su tipo, IP, vendor y estado, incluyendo uno marcado como OFFLINE y otro como UNKNOWN.

Hallazgos relevantes mapeados a STRIDE

Habíamos planteado cuatro hipótesis de riesgo antes de empezar, y la evidencia recogida las confirma de forma concreta y no solo teórica.

La primera es Information Disclosure sobre el contenido en tránsito. Quedó demostrada con la captura en Wireshark del POST hacia /api/scripts/simulate, donde el JSON completo con deviceId, scriptName y parameters viaja legible sin ningún tipo de cifrado. Cualquier persona en la misma red, sin necesidad de comprometer el servidor ni el navegador, puede leer exactamente qué se está enviando al backend.

La segunda es Information Disclosure sobre headers y versión del servidor. Nmap identificó nginx 1.30.4 en el primer escaneo, y ZAP confirmó el mismo dato de forma independiente a través del encabezado Server. Esa versión expuesta le facilita a un atacante buscar vulnerabilidades conocidas específicas de esa versión de Nginx sin tener que adivinar qué software corre detrás del puerto 80.

La tercera es Repudiation por falta de correlación temporal antes de tener logs claros. Mientras el backend estaba caído, los únicos síntomas visibles eran los 502 en el navegador, que por sí solos no dicen quién generó la petición ni por qué falló. Solo cuando Juan Pablo cruzó el journalctl con las peticiones del access.log fue posible atribuir cada error a una causa concreta y a un cliente concreto identificado por IP. Además, como el simulador de scripts no tiene autenticación, cualquier acción que se ejecute ahí queda asociada únicamente a una IP de origen y no a un usuario

identificable, lo que en un entorno real dificultará bastante investigar quién ejecutó qué script.

La cuarta es Tampering por ausencia de TLS. Como todo el tráfico entre el navegador y el servidor va sin cifrar ni firmar, no solo se puede leer el contenido de las peticiones como en el primer punto, sino que en teoría también podría modificarse en tránsito sin que ni el cliente ni el servidor tengan forma de detectarlo. La falta de HTTPS que dejamos a propósito para este laboratorio es la causa raíz común detrás de este riesgo y del primero.

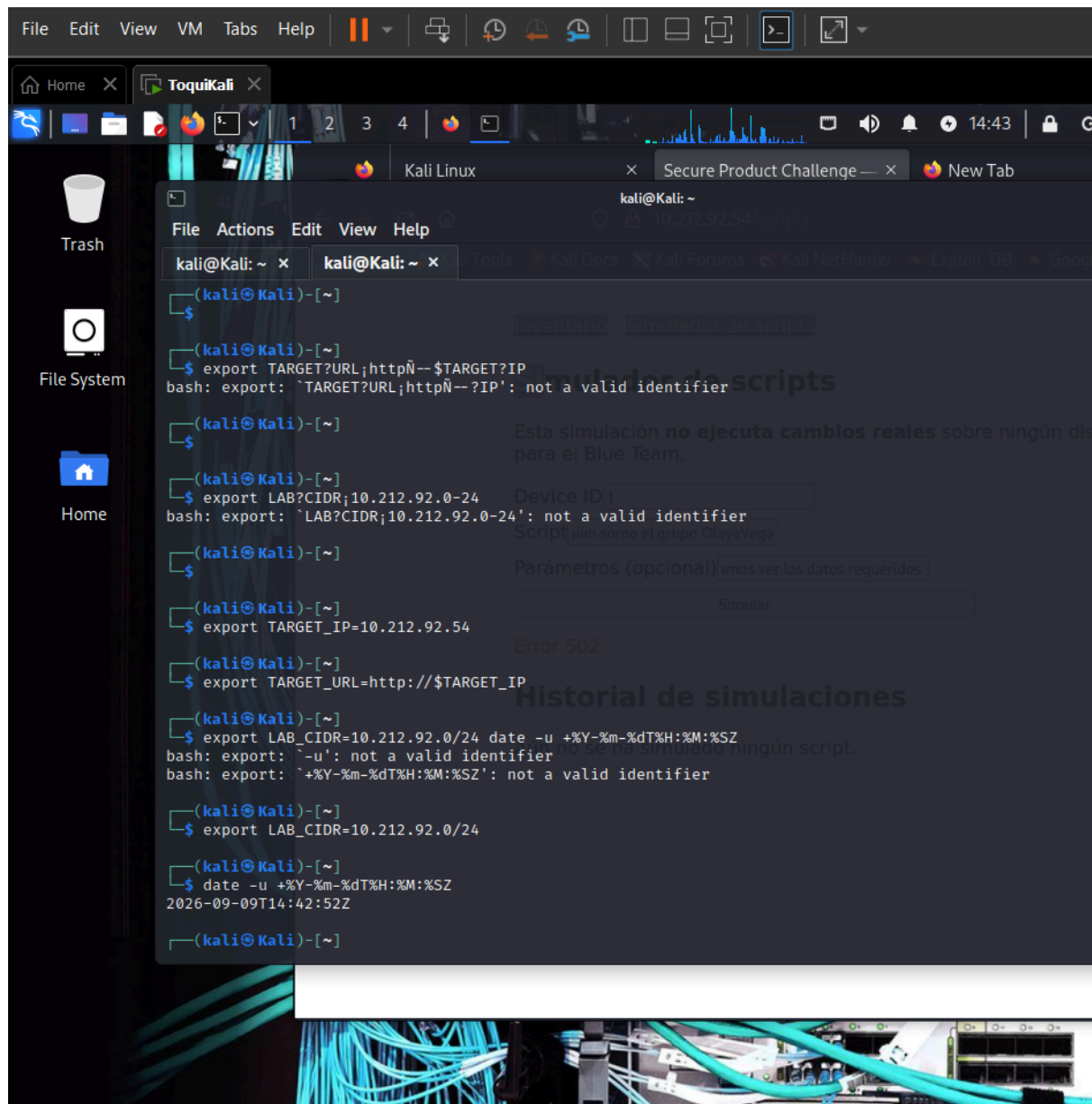
Corrección aplicada

A la fecha de este informe todavía no hicimos el retest, así que lo que sigue queda planteado como el trabajo pendiente para la Fase E. En Nginx hace falta desactivar `server_tokens` para dejar de exponer la versión exacta en el header `Server`, agregar los encabezados de seguridad que ZAP marcó como ausentes, es decir `Content-Security-Policy` y una directiva contra clickjacking como `X-Frame-Options` o `frame-ancestors`, además de `X-Content-Type-Options` en `nosniff`. También queda pendiente desactivar `autoindex` para evitar que se pueda listar el contenido de directorios si en algún momento se llega a exponer una carpeta sin `index`, y bloquear o devolver un 404 controlado en rutas que no deberían ser alcanzables desde afuera, como la que aparentemente cae por accidente en el fallback del SPA. Todo esto no resuelve por sí solo el problema de fondo, que es la falta de cifrado, pero sí cierra buena parte de la superficie de reconocimiento que explotamos en este laboratorio.

Conclusión

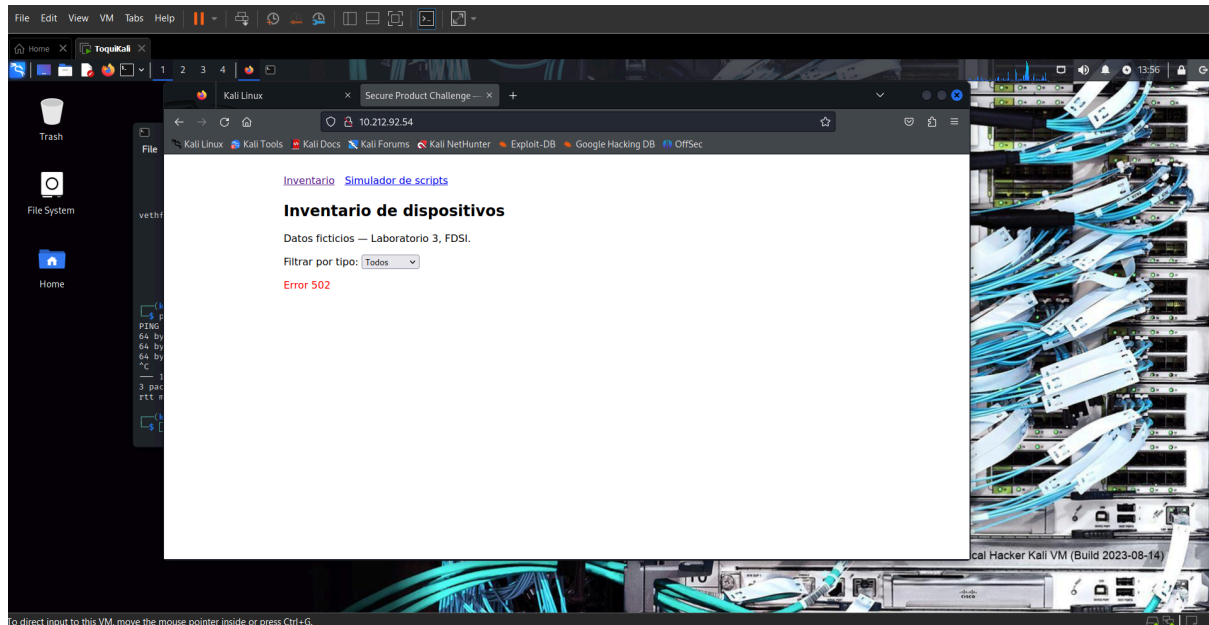
Este laboratorio dejó bastante claro que el riesgo más grande del despliegue no estaba en una vulnerabilidad exótica sino en decisiones básicas que se dejaron pendientes a propósito: no hay HTTPS, no hay autenticación y el servidor se identifica solo. Eso es justo lo que le da sentido al Laboratorio 4, donde toca resolver autenticación y roles para que acciones como usar el simulador de scripts queden atadas a un usuario real y no a una IP anónima, y agregar HTTPS para cerrar tanto el Information Disclosure como el riesgo de Tampering que identificamos acá. Lo que hicimos en este laboratorio fue justamente levantar esa lista de riesgos abiertos con evidencia real detrás, para que el siguiente paso no sea implementar seguridad en abstracto sino cerrar puntos específicos que ya sabemos que están expuestos.

Imágenes del laboratorio:



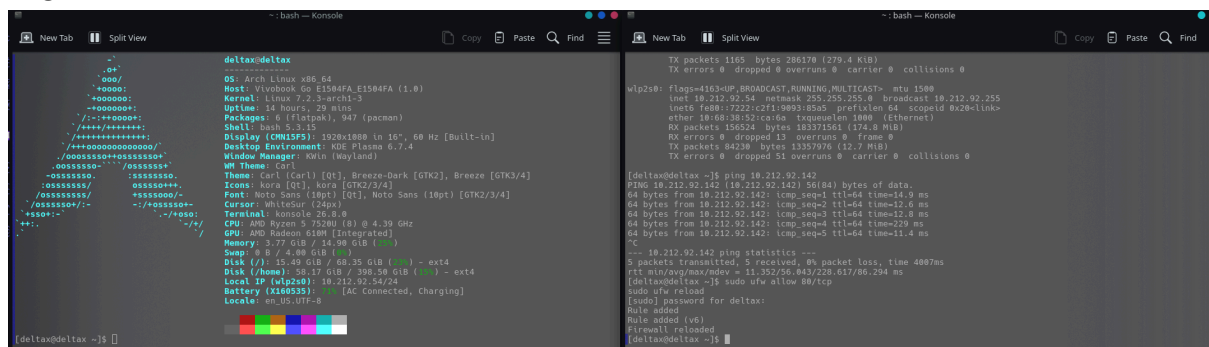
Configuración inicial de las variables de entorno `TARGET_IP`, `TARGET_URL` y `LAB_CIDR`, y registro de la hora de inicio con `date -u`, tal como pide la guía antes de ejecutar cualquier prueba contra el objetivo.

Olaya: Red Team



Vista inicial de la aplicación desde el navegador de Kali, rol Red Team: al entrar al Inventario de dispositivos aparece Error 502 porque en ese momento el backend todavía no estaba desplegado detrás de Nginx. Sin embargo después se corrió el backend para que todo corriera de forma correcta.

Vega: Blue Team



Estación de Juan Pablo Vega como Blue Team, corriendo Arch Linux, desde donde se hizo el monitoreo del servidor y la revisión de logs mientras yo ejecutaba las pruebas de reconocimiento.

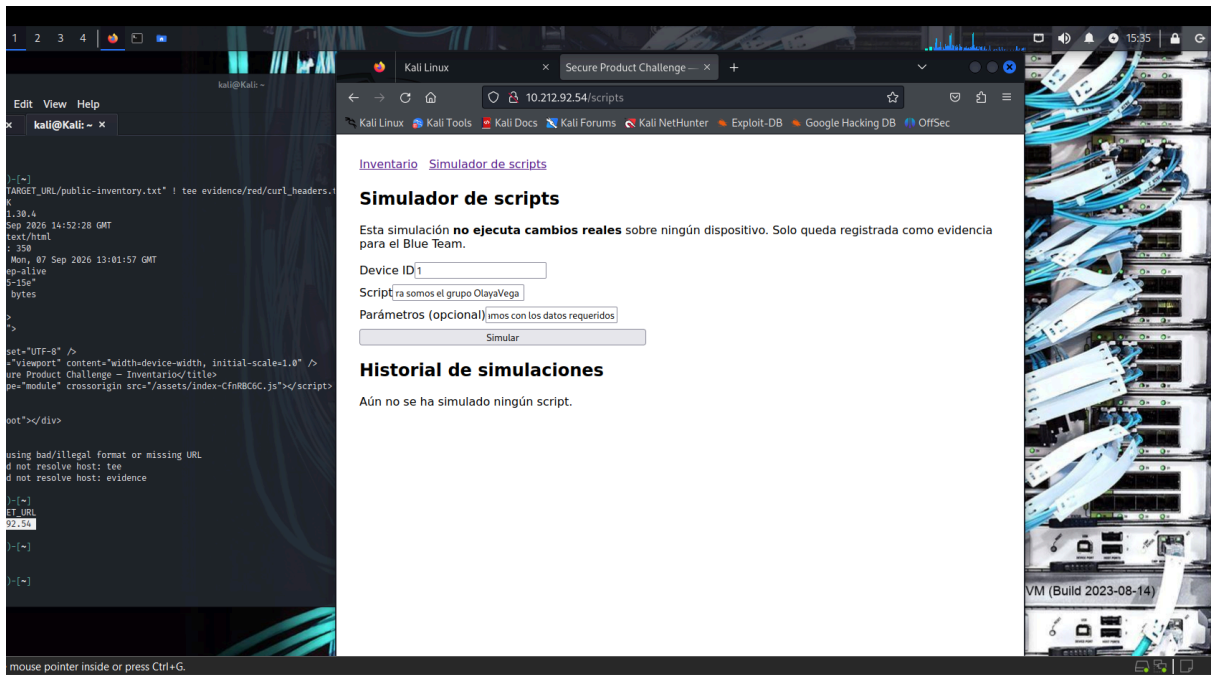
ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

No.	Time	Delta	Source	Destination	Protocol	Length	Info
774	31.570538079		10.212.92.142	10.212.92.54	HTTP	483	GET / HTTP/1.1
776	31.570752635	0.000214556	10.212.92.54	10.212.92.142	HTTP	246	HTTP/1.1 304 Not Modified
817	32.356754838	0.786002203	10.212.92.142	10.212.92.54	HTTP	364	GET /api/devices HTTP/1.1
818	32.357275960	0.000521122	10.212.92.54	10.212.92.142	HTTP	388	HTTP/1.1 502 Bad Gateway (text/html)

> Frame 774: Packet, 483 bytes on wire (3864 bits), 483 bytes captured (3864 bits) on interface wlp2s0, id 0	0000 10 68 30 52 c0 6a 50 fe 8c 01 fa 13 08 00 45 00	h0R jPE-
> Ethernet II, Src: AzureWaveTec_01:fa:13 (50:fe:0c:01:fa:13), Dst: AzureWaveTec_52:ca:6a (10:68:38:52:ca:6a)	0010 01 d5 79 43 40 00 40 06 f1 73 0a d4 5c 8e 0a d4	yC@ : s \..
> Internet Protocol Version 4, Src: 10.212.92.142, Dst: 10.212.92.54	0020 5c 36 d5 4c 00 50 ab e5 f3 66 08 3c 50 5e 80 18	\6 L P : : f <P ..
> Transmission Control Protocol, Src Port: 54604, Dst Port: 80, Seq: 1, Ack: 1, Len: 417	0030 01 f6 93 e7 00 00 01 01 08 0a 3e 30 af 5b 05 3f	>0 [?
> Hypertext Transfer Protocol	0040 44 cf 47 45 54 2b 2f 20 48 54 54 5b 2f 31 2e 31	0 GET / HTTP/1.1
	0050 0d 0a 48 6f 73 74 3a 20 31 30 2e 32 31 32 2e 39	: Host: 10.212.9
	0060 32 2e 35 34 0d 0a 55 73 65 72 2d 41 67 65 6e 74	2.54: Us er-Agent
	0070 3a 20 40 6f 7a 69 6c 6c 61 2f 35 2e 30 20 28 58	: Mozilla/5.0 (X
	0080 31 31 3b 20 4c 69 6e 75 78 28 70 38 36 5f 36 34	11; Linux x86_64
	0090 3b 20 72 76 3a 31 30 39 2e 30 29 20 47 65 63 6b	; rv:109.0) Gecko
	00a0 6f 2f 32 30 31 30 30 31 30 31 20 46 69 72 65 66	o/201001 01 Firef
	00b0 6f 70 2f 31 31 35 2e 30 0d 0a 41 63 63 65 70 74	ox/115.0 - Accept
	00c0 3a 20 74 65 78 74 2f 68 74 6d 6c 2c 61 70 70 6c	: text/h tml,appl
	00d0 69 63 61 74 69 6f 6e 2f 78 68 74 6d 6c 2b 78 6d	ication/ xhtml+xml
	00e0 6c 2c 61 70 70 6c 69 63 61 74 69 6f 6e 2f 78 6d	l,applic ation/xm
	00f0 6c 3b 71 3d 30 2e 39 2c 69 6d 61 67 65 2f 61 76	l;q=0.9, image/av
	0100 69 66 2c 69 6d 61 67 65 2f 77 65 62 70 2c 2a 2f	lf,image /webp,*/
	0110 2a 3b 71 3d 30 2e 38 0d 0a 41 63 63 65 70 74 2d	*;q=0.8 - Accept-
	0120 4c 61 6e 67 75 61 67 65 3a 28 65 6e 2d 55 53 2c	Language : en-US,
	0130 65 6e 3b 71 3d 30 2e 35 0d 0a 41 63 63 65 70 74	en;q=0.5 - Accept
	0140 2d 45 6e 63 6f 64 69 6e 67 3a 20 67 7a 69 70 2c	-Encodin g: gzip,
	0150 20 64 65 66 6c 61 74 65 0d 0a 43 6f 6e 6e 65 63	deflate - Connec
	0160 74 69 6f 6e 3a 28 6b 65 65 78 2d 61 6c 69 76 65	tion: ke ep-alive
	0170 0d 0a 55 70 67 72 61 64 65 2d 49 6e 73 65 63 75	- Upgrad e-Insecu
	0180 72 65 2d 52 65 71 75 65 73 74 73 3a 20 31 0d 0a	re-Reque sts: 1..
	0190 49 66 2d 4d 6f 64 69 66 69 65 64 2d 53 69 6e 63	If-Modif ied-Sinc
	01a0 65 3a 20 4d 6f 6e 2c 20 30 37 20 53 65 70 28 32	e: Mon, 07 Sep 2
	01b0 30 32 36 20 31 33 3a 30 31 3a 35 37 20 47 4d 54	026 13:0 1:57 GMT
	01c0 0d 0a 49 66 2d 4e 6f 6e 65 2d 4d 61 74 63 68 3a	- If-Non e-Match:
	01d0 20 22 36 61 39 65 62 35 63 35 2d 31 35 65 22 6d	"6a9eb5 c5-15e"

Paso 11 de la guía, demostrar visibilidad de HTTP: mientras Blue Team captura tráfico en el servidor con tcpdump, yo como Red Team solicito la página raíz y la API. En Wireshark se ve el GET / respondido con 304 Not Modified y el GET /api/devices fallando con 502 Bad Gateway, porque el backend aún no estaba levantado.

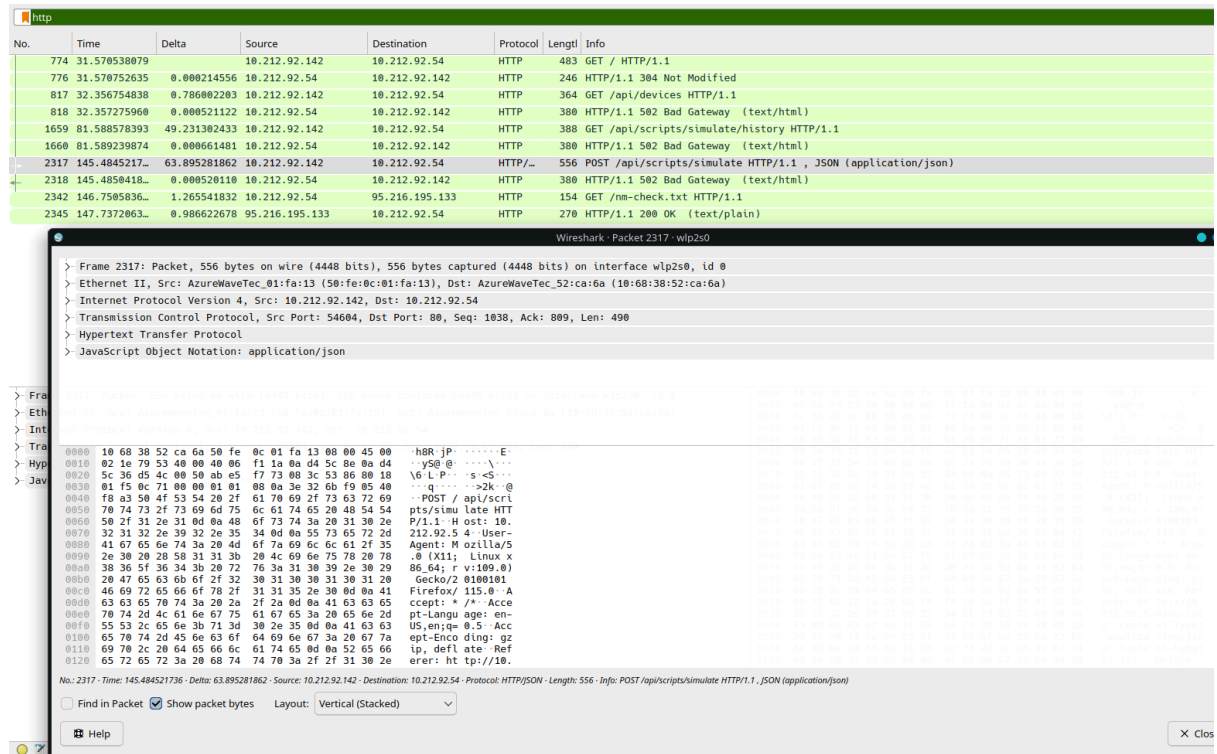


The screenshot shows a Kali Linux environment. On the left, a terminal window displays the output of a script execution, showing a successful GET request to /public-inventory.txt. On the right, a web browser window shows a page titled "Simulador de scripts" (Script Simulator). The page contains a form for simulating a script execution, with fields for "Device ID" (set to 1) and "Script" (set to "ra somos el grupo OlayaVega"). The "Simular" button is visible. Below the form, there is a section titled "Historial de simulaciones" (Simulation History) which currently shows "Aún no se ha simulado ningún script." (No script has been simulated yet).

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

Formulario del Simulador de scripts en la aplicación: al llenar Device ID y un mensaje de prueba y darle a Simular, se dispara el POST que capturamos en Wireshark en el paso anterior.



The screenshot displays the Wireshark network protocol analyzer. The top pane shows a list of captured packets, with packet 2317 highlighted. The bottom pane provides a detailed view of this packet, showing the Ethernet II, Internet Protocol Version 4, and Hypertext Transfer Protocol layers. The JSON body of the POST request is visible in the packet bytes pane.

No.	Time	Delta	Source	Destination	Protocol	Length	Info
774	31.570538079		10.212.92.142	10.212.92.54	HTTP	483	GET / HTTP/1.1
776	31.570752635	0.000214556	10.212.92.142	10.212.92.142	HTTP	246	HTTP/1.1 304 Not Modified
817	32.356754838	0.786002203	10.212.92.142	10.212.92.54	HTTP	364	GET /api/devices HTTP/1.1
818	32.357275968	0.000521122	10.212.92.54	10.212.92.142	HTTP	380	HTTP/1.1 502 Bad Gateway (text/html)
1659	81.588578393	49.231302433	10.212.92.142	10.212.92.54	HTTP	388	GET /api/scripts/simulate/history HTTP/1.1
1660	81.589239874	0.000661481	10.212.92.54	10.212.92.142	HTTP	380	HTTP/1.1 502 Bad Gateway (text/html)
2317	145.4845217...	63.895281862	10.212.92.142	10.212.92.54	HTTP/...	556	POST /api/scripts/simulate HTTP/1.1, JSON (application/json)
2318	145.4850418...	0.000520110	10.212.92.54	10.212.92.142	HTTP	380	HTTP/1.1 502 Bad Gateway (text/html)
2342	146.7505836...	1.265541832	10.212.92.54	95.216.195.133	HTTP	154	GET /nm-check.txt HTTP/1.1
2345	147.7372063...	0.986622678	95.216.195.133	10.212.92.54	HTTP	270	HTTP/1.1 200 OK (text/plain)

Frame 2317: Packet, 556 bytes on wire (4448 bits), 556 bytes captured (4448 bits) on interface wlp2s0, id 0

- Ethernet II, Src: AzureWaveTec_01:fa:13 (58:fe:0c:01:fa:13), Dst: AzureWaveTec_52:ca:6a (10:68:38:52:ca:6a)
- Internet Protocol Version 4, Src: 10.212.92.142, Dst: 10.212.92.54
- Transmission Control Protocol, Src Port: 54604, Dst Port: 80, Seq: 1038, Ack: 809, Len: 490
- Hypertext Transfer Protocol
- JavaScript Object Notation: application/json

```

0000  10 68 38 52 ca 6a 50 fe 0c 01 fa 13 08 00 45 00  -hBR.jp.....E
0010  02 1e 79 53 40 00 40 06 f1 1a 0a d4 5c 8e 0a d4  -ys@.....\
0020  5c 36 d5 4c 00 50 ab e5 f7 73 08 3c 53 86 00 18  -\6.L.P...s<S...
0030  01 15 0c 71 00 00 01 01 00 0a 3e 32 6b f9 05 40  -q...>2K...@
0040  f8 a3 50 4f 53 54 20 2f 61 70 69 2f 73 63 72 69  -POST /api/scri
0050  70 74 73 2f 73 69 6d 75 6c 61 74 65 20 48 54 54  -pts/simu late HT
0060  50 2f 31 2e 31 0d 0a 48 6f 73 74 3a 20 31 39 2e  -P/1.1 Host: 10.
0070  32 31 32 2e 39 32 2e 35 34 0d 0a 55 73 65 72 2d  -212.92.54 User-
0080  41 67 65 6e 74 3a 20 4d 6f 7a 69 6e 6c 61 2f 35  -Agent: Mozilla/5
0090  2e 30 20 28 58 31 31 3b 20 4c 69 6e 75 78 20 78  -.0 (X11; Linux x
00a0  38 36 5f 36 34 3b 20 72 76 3a 31 30 39 2e 30 29  -86.64; rv:109.0)
00b0  20 47 65 63 6b 6f 2f 32 30 31 30 30 31 30 31 20  -Gecko/2.0.100101
00c0  46 69 72 65 66 6f 78 2f 31 31 35 2e 30 0d 0a 41  -Firefox/115.0 A
00d0  63 63 65 70 74 3a 20 2a 2f 2a 0d 0a 41 63 63 65  -ccept: /* Acce
00e0  70 74 2d 4c 61 6e 67 75 61 67 65 3a 20 65 6e 2d  -pt-Langu age: en-
00f0  55 53 2c 65 6e 3b 71 3d 30 2e 35 0d 0a 41 63 63  -US,en;q=0.5 Acc
0100  65 70 74 2d 45 6e 63 6f 64 69 6e 67 3a 20 67 7a  -ept-Enco ding: g2
0110  69 70 2c 20 64 65 66 6c 61 74 65 0d 0a 52 65 66  -ip, deflate Ref
0120  65 72 65 72 3a 20 68 74 74 70 3a 2f 2f 31 30 2e  -errer: ht tp://10.
  
```

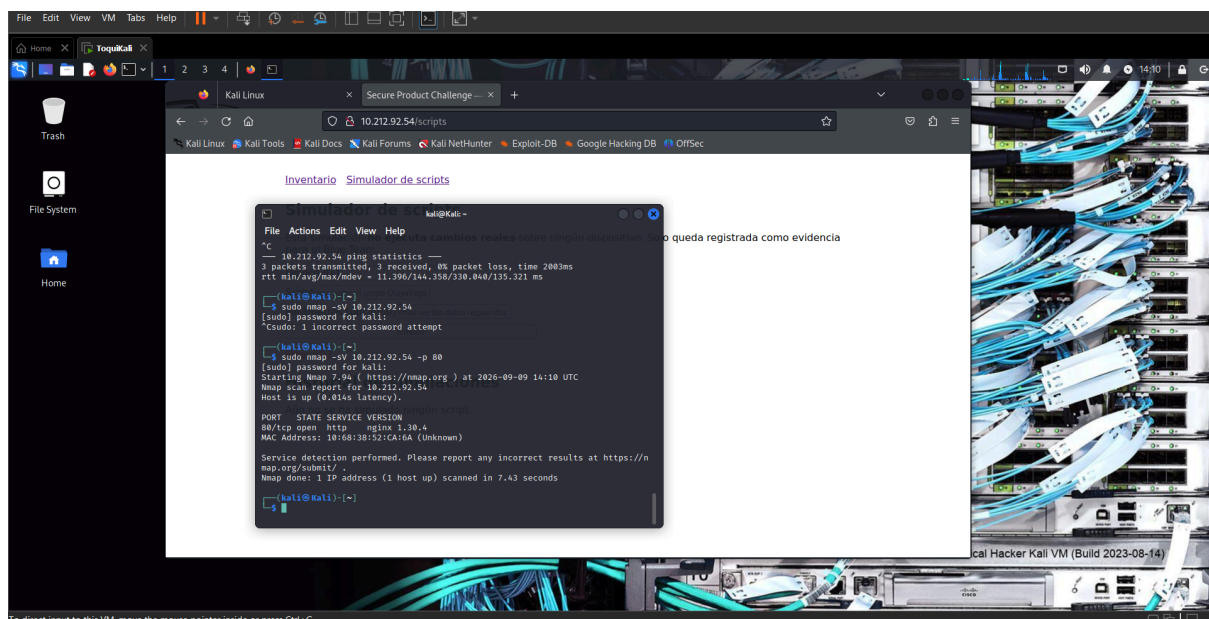
No.: 2317 - Time: 145.484521736 - Delta: 63.895281862 - Source: 10.212.92.142 - Destination: 10.212.92.54 - Protocol: HTTP/JSON - Length: 556 - Info: POST /api/scripts/simulate HTTP/1.1, JSON (application/json)

☐ Find in Packet ☒ Show packet bytes Layout: Vertical (Stacked)

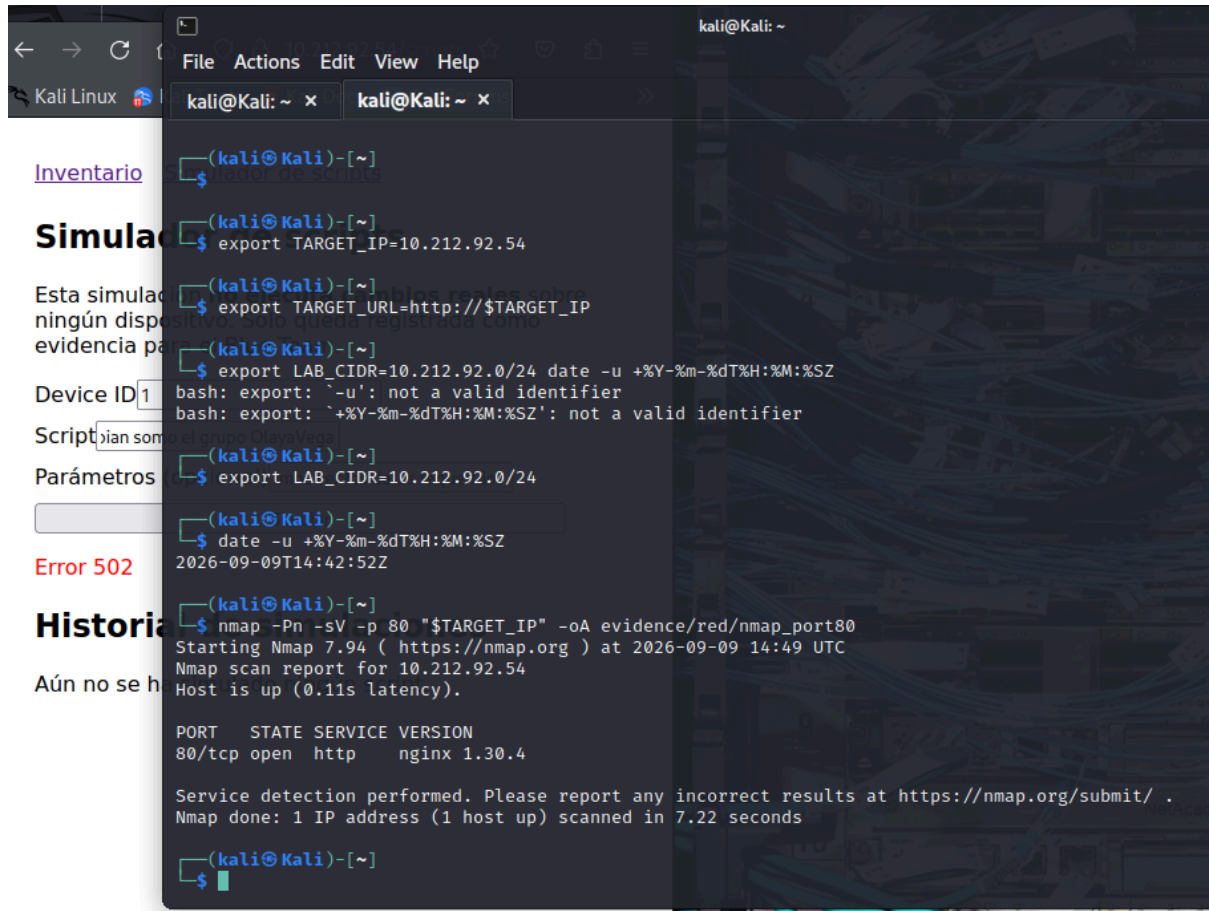
Detalle en hexadecimal del paquete POST capturado en Wireshark, con el body en JSON completamente legible: deviceId, scriptName y parameters viajan en texto plano por no usar HTTPS, confirmando la hipótesis H1 de Information Disclosure

sobre contenido en tránsito.

Key: parameters		
00f0	55 53 2c 65 6e 3b 71 3d 30 2e 35 0d 0a 41 63 63	US,en;q=0.5·Acc
0100	65 70 74 2d 45 6e 63 6f 64 69 6e 67 3a 20 67 7a	pt-Enco ding: gz
0110	69 70 2c 20 64 65 66 6c 61 74 65 0d 0a 52 65 66	ip, defl ate·Ref
0120	65 72 65 72 3a 20 68 74 74 70 3a 2f 2f 31 30 2e	erer: ht tp://10.
0130	32 31 32 2e 39 32 2e 35 34 2f 73 63 72 69 70 74	212.92.5 4/script
0140	73 0d 0a 43 6f 6e 74 65 6e 74 2d 54 79 70 65 3a	s·Conte nt-Type:
0150	20 61 70 70 6c 69 63 61 74 69 6f 6e 2f 6a 73 6f	applica tion/jso
0160	6e 0d 0a 43 6f 6e 74 65 6e 74 2d 4c 65 6e 67 74	n·Conte nt-Lengt
0170	68 3a 20 31 32 35 0d 0a 4f 72 69 67 69 6e 3a 20	h: 125· Origin:
0180	68 74 74 70 3a 2f 2f 31 30 2e 32 31 32 2e 39 32	http://1 0.212.92
0190	2e 35 34 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e 3a	.54·Con nection:
01a0	20 6b 65 65 70 2d 61 6c 69 76 65 0d 0a 0d 0a 7b	keep-al ive···{
01b0	22 64 65 76 69 63 65 49 64 22 3a 22 31 22 2c 22	"deviceI d":"1","
01c0	73 63 72 69 70 74 4e 61 6d 65 22 3a 22 48 6f 6c	scriptNa me":"Hol
01d0	61 20 70 72 6f 66 65 20 46 61 62 69 61 6e 20 73	a profe Fabian s
01e0	6f 6d 6f 20 65 6c 20 67 72 75 70 6f 20 4f 6c 61	om el g rupo Ola
01f0	79 61 56 65 67 61 22 2c 22 70 61 72 61 6d 65 74	yaVega", "paramet
0200	65 72 73 22 3a 22 59 20 6c 6f 67 72 61 6d 6f 73	ers":"Y logramos
0210	20 76 65 72 20 6c 6f 73 20 64 61 74 6f 73 20 72	ver los datos r
0220	65 71 75 65 72 69 64 6f 73 20 22 7d	equerido s"} }



Paso 8 de la guía, reconocimiento de puerto y servicio: prueba inicial de conectividad contra el objetivo antes de lanzar el escaneo con Nmap.



The screenshot shows a Kali Linux terminal window with the following commands and output:

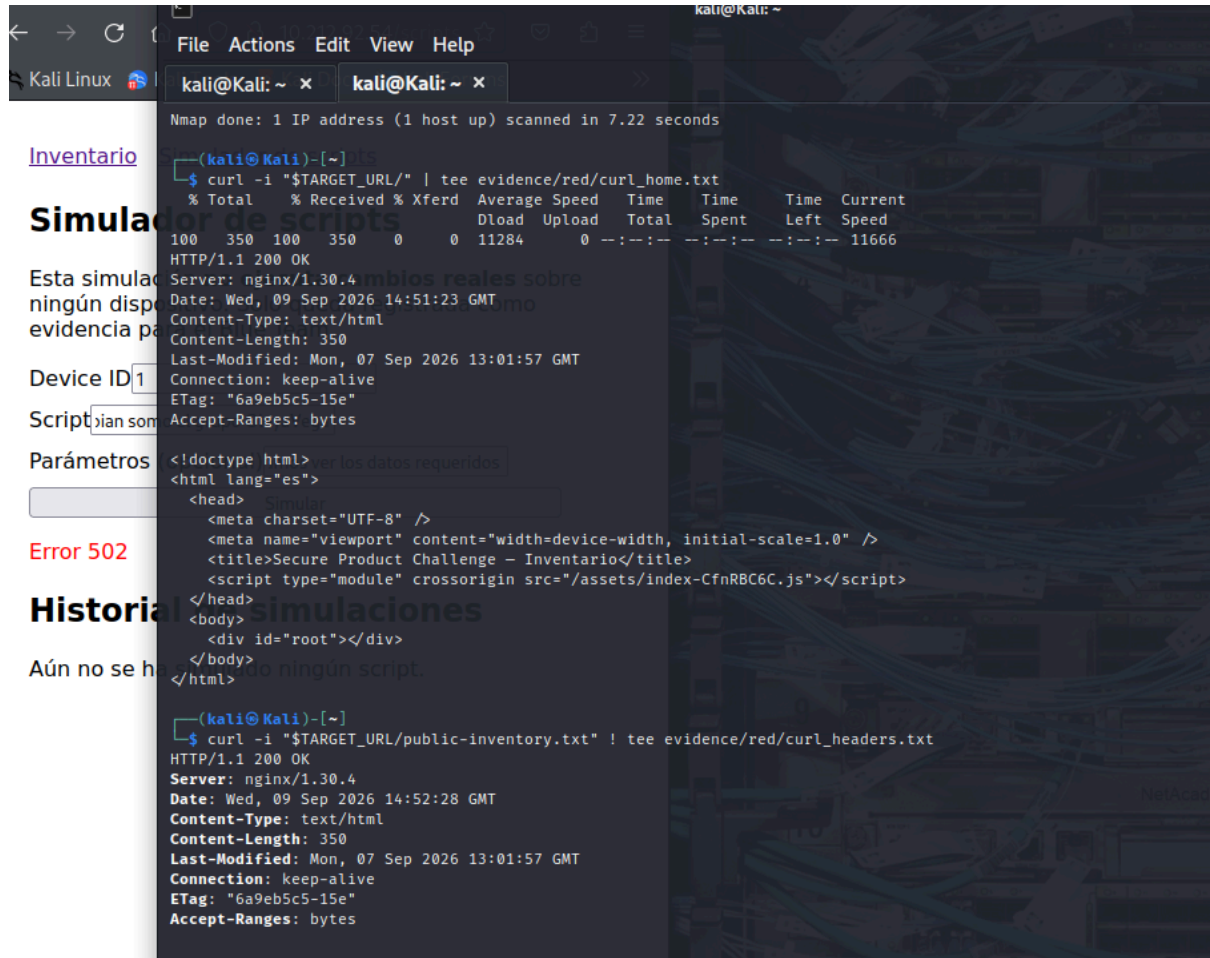
```
(kali@kali)-[~]
$ export TARGET_IP=10.212.92.54
(kali@kali)-[~]
$ export TARGET_URL=http://$TARGET_IP
(kali@kali)-[~]
$ export LAB_CIDR=10.212.92.0/24 date -u +%Y-%m-%dT%H:%M:%SZ
bash: export: `~u': not a valid identifier
bash: export: `+%Y-%m-%dT%H:%M:%SZ': not a valid identifier
(kali@kali)-[~]
$ export LAB_CIDR=10.212.92.0/24
(kali@kali)-[~]
$ date -u +%Y-%m-%dT%H:%M:%SZ
2026-09-09T14:42:52Z
(kali@kali)-[~]
$ nmap -Pn -sV -p 80 "$TARGET_IP" -oA evidence/red/nmap_port80
Starting Nmap 7.94 ( https://nmap.org ) at 2026-09-09 14:49 UTC
Nmap scan report for 10.212.92.54
Host is up (0.11s latency).

PORT      STATE SERVICE VERSION
80/tcp    open  http    nginx 1.30.4

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.22 seconds
(kali@kali)-[~]
$
```

In the background, a web browser window is visible showing a page with the text "Error 502".

Escaneo con `nmap -Pn -sV -p 80 $TARGET_IP`: se identifica el puerto 80 abierto con el servicio HTTP y la versión exacta nginx 1.30.4, evidenciando la hipótesis H2 de Information Disclosure sobre headers y versión del servidor. De fondo se ve también el navegador con el Error 502 mencionado antes.



```

kali@Kali: ~
File Actions Edit View Help
kali@Kali: ~ x kali@Kali: ~ x
Nmap done: 1 IP address (1 host up) scanned in 7.22 seconds

(kali@Kali)-[~] $ curl -i "$TARGET_URL/" | tee evidence/red/curl_home.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           % Dload  % Upload   Dload  Upload   Total   Spent    Left   Speed
100    350    100    350      0      0  11284      0 --:--:-- --:--:-- --:--:-- 11666
HTTP/1.1 200 OK
Server: nginx/1.30.4
Date: Wed, 09 Sep 2026 14:51:23 GMT
Content-Type: text/html
Content-Length: 350
Last-Modified: Mon, 07 Sep 2026 13:01:57 GMT
Connection: keep-alive
ETag: "6a9eb5c5-15e"
Accept-Ranges: bytes

<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Secure Product Challenge – Inventario</title>
    <script type="module" crossorigin src="/assets/index-CfnRBC6C.js"></script>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>

(kali@Kali)-[~] $ curl -i "$TARGET_URL/public-inventory.txt" | tee evidence/red/curl_headers.txt
HTTP/1.1 200 OK
Server: nginx/1.30.4
Date: Wed, 09 Sep 2026 14:52:28 GMT
Content-Type: text/html
Content-Length: 350
Last-Modified: Mon, 07 Sep 2026 13:01:57 GMT
Connection: keep-alive
ETag: "6a9eb5c5-15e"
Accept-Ranges: bytes
  
```

Paso 8 continuado, curl -i "\$TARGET_URL/": la respuesta llega en texto plano por HTTP, con los headers Server nginx/1.30.4, Content-Type, Content-Length y ETag, seguidos del HTML completo del index de la aplicación, sin ningún cifrado de por medio.

```
(kali@kali)-[~]
└─$ curl -i "$TARGET_URL/public-inventory.txt" ! tee evidence/red/curl_headers.txt
HTTP/1.1 200 OK
Server: nginx/1.30.4
Date: Wed, 09 Sep 2026 14:52:28 GMT
Content-Type: text/html
Content-Length: 350
Last-Modified: Mon, 07 Sep 2026 13:01:57 GMT
Connection: keep-alive
ETag: "6a9eb5c5-15e"
Accept-Ranges: bytes

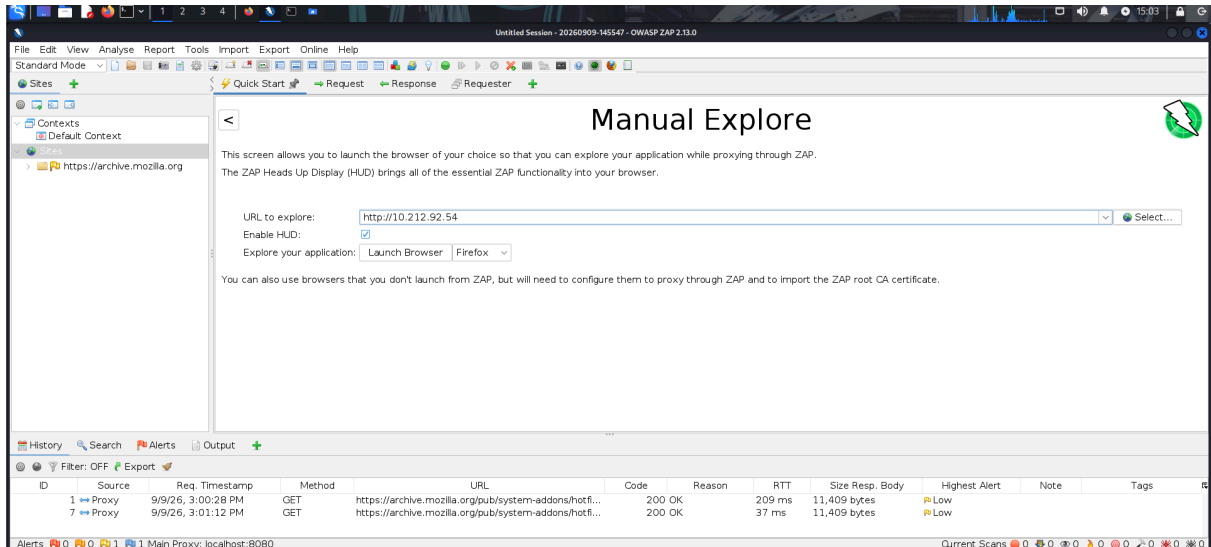
<!doctype html>
<html lang="es">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Secure Product Challenge – Inventario</title>
    <script type="module" crossorigin src="/assets/index-CfnRBC6C.js"></script>
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
curl: (3) URL using bad/illegal format or missing URL
curl: (6) Could not resolve host: tee
curl: (6) Could not resolve host: evidence

(kali@kali)-[~]
└─$
```

Continuación del Paso 8, `curl -i "$TARGET_URL/public-inventory.txt"`: esperaba un archivo con datos de inventario en texto plano, pero la respuesta trae el mismo `Content-Length 350` y el mismo HTML del shell de la aplicación, probablemente por el `try_files` de Nginx cayendo a `index.html`. También se ve un error de sintaxis en el comando, usé `!` en vez de `|` antes de `tee`, que generó los mensajes de bash de abajo sin afectar la respuesta ya capturada.

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

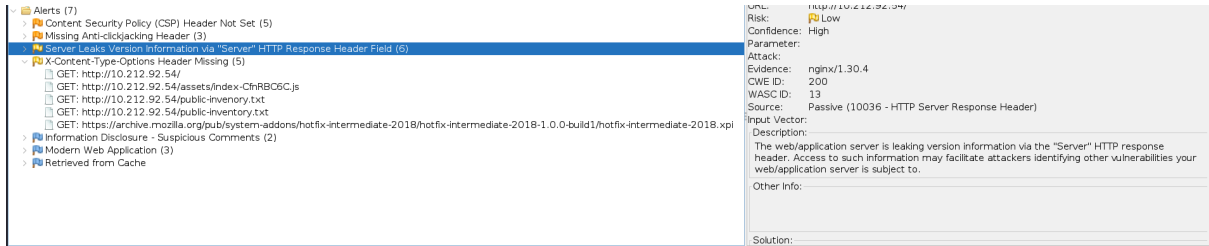


Paso 10 de la guía, inspección pasiva con OWASP ZAP: se configura Manual Explore apuntando únicamente a TARGET_URL, sin ejecutar Active Scan, tal como exige la guía. A partir de acá navegamos la página y el archivo público dejando que ZAP registre los hallazgos pasivos.



ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO

DEPARTAMENTO DE INGENIERÍA DE SISTEMAS



Alerts (7)

- Content Security Policy (CSP) Header Not Set (5)
- Missing Anti-clickjacking Header (3)
- Server Leaks Version Information via "Server" HTTP Response Header Field (6)**
 - X-Content-Type-Options Header Missing (5)
 - GET: http://10.212.92.54/
 - GET: http://10.212.92.54/assets/index-ChnRBO6C.js
 - GET: http://10.212.92.54/public-inventory.txt
 - GET: http://10.212.92.54/public-inventory.txt
 - GET: https://archive.mozilla.org/pub/system-addons/hotfix-intermediate-2018/hotfix-intermediate-2018-1.0.0-build1/hotfix-intermediate-2018.xpi
 - Information Disclosure - Suspicious Comments (2)
 - Modern Web Application (3)
 - Retrieved from Cache

URL: http://10.212.92.54/

Risk: Low

Confidence: High

Parameter:

Attack: nginx/1.30.4

Evidence: nginx/1.30.4

CWE ID: 200

WASC ID: 13

Source: Passive (10036 - HTTP Server Response Header)

Input Vector:

Description: The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.

Other Info:

Solution:



History Search Alerts Output WebSockets

Alerts (7)

- Content Security Policy (CSP) Header Not Set (5)
- Missing Anti-clickjacking Header (3)
- Server Leaks Version Information via "Server" HTTP Response Header Field (6)
- X-Content-Type-Options Header Missing (5)
- Information Disclosure - Suspicious Comments (2)**
 - GET: http://10.212.92.54/assets/index-ChnRBO6C.js
 - GET: http://10.212.92.54/assets/index-ChnRBO6C.js
- Modern Web Application (3)
- Retrieved from Cache

Information Disclosure - Suspicious Comments

URL: http://10.212.92.54/assets/index-ChnRBO6C.js

Risk: Informational

Confidence: Low

Parameter:

Attack: select

Evidence: 200

CWE ID: 200

WASC ID: 13

Source: Passive (10027 - Information Disclosure - Suspicious Comments)

Input Vector:

Description: The response appears to contain suspicious comments which may help an attacker. Note: Matches made within script blocks or files are against the entire content not only comments.

Other Info: The following pattern was used: \bSELECT\b and was detected 3 times, the first in the element starting with: "<+([o].replace(" at new ", " at ");return e.displayName&&s.includes("<anonymous>")&&(s=s.replace("<anonymous>",e.displayName)),s");, see evidence field for the suspicious comment/snippet.

Solution:

Alerts 0 2 2 3 Main Proxy: localhost:8080

Current Scans 0 0 0 0 0 0 0 0 0 0 0 0



History Search Alerts Output WebSockets

Alerts (7)

- Content Security Policy (CSP) Header Not Set (5)
- Missing Anti-clickjacking Header (3)
- Server Leaks Version Information via "Server" HTTP Response Header Field (6)
- X-Content-Type-Options Header Missing (5)
- Information Disclosure - Suspicious Comments (2)
- Modern Web Application (3)**
 - GET: http://10.212.92.54/
 - GET: http://10.212.92.54/public-inventory.txt
 - GET: http://10.212.92.54/public-inventory.txt
- Retrieved from Cache

Modern Web Application

URL: http://10.212.92.54/

Risk: Informational

Confidence: Medium

Parameter:

Attack: <script type="module" crossorigin src="/assets/index-ChnRBO6C.js"></script>

Evidence: 200

CWE ID: 10109

WASC ID: 10109

Source: Passive (10109 - Modern Web Application)

Input Vector:

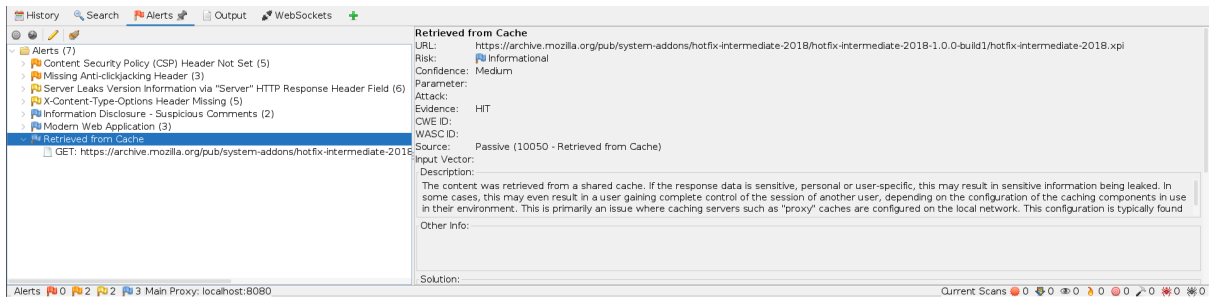
Description: The application appears to be a modern web application. If you need to explore it automatically then the Ajax Spider may well be more effective than the standard one.

Other Info: No links have been found while there are scripts, which is an indication that this is a modern web application.

Solution:

Alerts 0 2 2 3 Main Proxy: localhost:8080

Current Scans 0 0 0 0 0 0 0 0 0 0 0 0



History Search Alerts Output WebSockets

Alerts (7)

- Content Security Policy (CSP) Header Not Set (5)
- Missing Anti-clickjacking Header (3)
- Server Leaks Version Information via "Server" HTTP Response Header Field (6)
- X-Content-Type-Options Header Missing (5)
- Information Disclosure - Suspicious Comments (2)
- Modern Web Application (3)
- Retrieved from Cache**
 - GET: https://archive.mozilla.org/pub/system-addons/hotfix-intermediate-2018/hotfix-intermediate-2018-1.0.0-build1/hotfix-intermediate-2018.xpi

Retrieved from Cache

URL: https://archive.mozilla.org/pub/system-addons/hotfix-intermediate-2018/hotfix-intermediate-2018-1.0.0-build1/hotfix-intermediate-2018.xpi

Risk: Informational

Confidence: Medium

Parameter:

Attack: HIT

Evidence: HIT

CWE ID: 10050

WASC ID: 10050

Source: Passive (10050 - Retrieved from Cache)

Input Vector:

Description: The content was retrieved from a shared cache. If the response data is sensitive, personal or user-specific, this may result in sensitive information being leaked. In some cases, this may even result in a user gaining complete control of the session of another user, depending on the configuration of the caching components in use in their environment. This is primarily an issue where caching servers such as "proxy" caches are configured on the local network. This configuration is typically found

Other Info:

Solution:

Alerts 0 2 2 3 Main Proxy: localhost:8080

Current Scans 0 0 0 0 0 0 0 0 0 0 0 0

Hallazgos pasivos que arrojó ZAP en el Paso 10: Content Security Policy Header Not Set y Missing Anti-clickjacking Header en varias rutas, Server Leaks Version Information mostrando nginx/1.30.4 como evidencia, X-Content-Type-Options Header Missing, Information Disclosure por comentarios sospechosos en el bundle de JavaScript minificado, que puede ser un falso positivo del patrón SELECT, y la alerta informativa Modern Web Application por el uso de un script tipo módulo.

Revisión logs

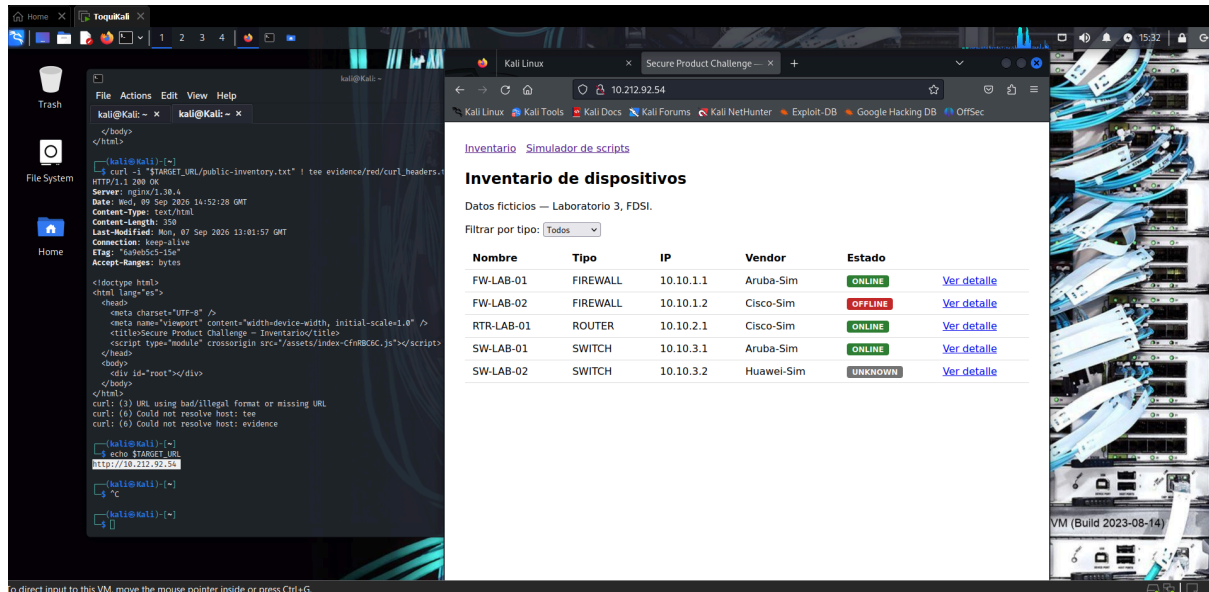
```
2.54/scripts" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:06:34 -0500] "GET /public-inventory.txt HTTP/1.1" 200 350 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:06:46 -0500] "GET /favicon.ico HTTP/1.1" 200 350 "https://10.212.92.54/public-inventory.txt" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:06:49 -0500] "GET /public-inventory.txt HTTP/1.1" 200 350 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:06:51 -0500] "GET /favicon.ico HTTP/1.1" 200 350 "https://10.212.92.54/public-inventory.txt" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:16:13 -0500] "GET /api/devices HTTP/1.1" 502 157 "http://10.212.92.54/" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:16:34 -0500] "GET /public-inventory.txt HTTP/1.1" 200 350 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:16:38 -0500] "GET /api/scripts/simulate/history HTTP/1.1" 502 157 "http://10.212.92.54/scripts" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:16:45 -0500] "POST /api/scripts/simulate HTTP/1.1" 502 157 "http://10.212.92.54/scripts" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:16:57 -0500] "GET /api/devices HTTP/1.1" 502 157 "http://10.212.92.54/" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
0.212.92.142 - - [09/Sep/2026:10:17:01 -0500] "GET /api/devices?type=FIREWALL HTTP/1.1" 502 157 "http://10.212.92.54/" "Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0"
```

Paso 12 de la guía, revisión de telemetría del servidor: Juan Pablo como Blue Team revisa el access.log de Nginx y encuentra las peticiones GET a /public-inventory.txt y /favicon.ico, además de los intentos contra /api/scripts/simulate y /api/devices que respondieron 502 mientras el backend aún no estaba desplegado.

`sudo tail -n 30 /var/log/nginx/error.log` → Errores porque no habíamos desplegado el backend

```
[deltax@deltax ~]$ sudo journalctl -u nginx --since '20 minutes ago' --no-pager
Sep 09 10:16:13 deltax nginx[5089]: 2026/09/09 10:16:13 [error] 5089#5089: *45 connect() failed (111: Connection refused) while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/devices HTTP/1.1", upstream: "http://127.0.0.1:8081/api/devices", host: "10.212.92.54", referrer: "http://10.212.92.54/"
Sep 09 10:16:13 deltax nginx[5089]: 2026/09/09 10:16:13 [error] 5089#5089: *45 connect() failed (111: Connection refused) while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/devices HTTP/1.1", upstream: "http://[::1]:8081/api/devices", host: "10.212.92.54", referrer: "http://10.212.92.54/"
Sep 09 10:16:38 deltax nginx[5089]: 2026/09/09 10:16:38 [error] 5089#5089: *45 connect() failed (111: Connection refused) while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/scripts/simulate/history HTTP/1.1", upstream: "http://127.0.0.1:8081/api/scripts/simulate/history", host: "10.212.92.54", referrer: "http://10.212.92.54/scripts"
Sep 09 10:16:38 deltax nginx[5089]: 2026/09/09 10:16:38 [error] 5089#5089: *45 connect() failed (111: Connection refused) while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/scripts/simulate/history HTTP/1.1", upstream: "http://[::1]:8081/api/scripts/simulate/history", host: "10.212.92.54", referrer: "http://10.212.92.54/scripts"
Sep 09 10:16:45 deltax nginx[5089]: 2026/09/09 10:16:45 [error] 5089#5089: *45 no live upstreams while connecting to upstream, client: 10.212.92.142, server: localhost, request: "POST /api/scripts/simulate HTTP/1.1", upstream: "http://localhost/api/scripts/simulate", host: "10.212.92.54", referrer: "http://10.212.92.54/scripts"
Sep 09 10:16:57 deltax nginx[5089]: 2026/09/09 10:16:57 [error] 5089#5089: *45 connect() failed (111: Connection refused) while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/devices HTTP/1.1", upstream: "http://127.0.0.1:8081/api/devices", host: "10.212.92.54", referrer: "http://10.212.92.54/"
Sep 09 10:16:57 deltax nginx[5089]: 2026/09/09 10:16:57 [error] 5089#5089: *45 connect() failed (111: Connection refused) while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/devices HTTP/1.1", upstream: "http://[::1]:8081/api/devices", host: "10.212.92.54", referrer: "http://10.212.92.54/"
Sep 09 10:17:01 deltax nginx[5089]: 2026/09/09 10:17:01 [error] 5089#5089: *45 no live upstreams while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/devices?type=FIREWALL HTTP/1.1", upstream: "http://localhost/api/devices?type=FIREWALL", host: "10.212.92.54", referrer: "http://10.212.92.54/"
Sep 09 10:20:21 deltax nginx[5089]: 2026/09/09 10:20:21 [error] 5089#5089: *52 connect() failed (111: Connection refused) while connecting to upstream, client: 10.212.92.142, server: localhost, request: "GET /api/devices HTTP/1.1", upstream: "http://127.0.0.1:8081/api/devices", host: "10.212.92.54", referrer: "http://10.212.92.54/"
```

Continuación del Paso 12 con `journalctl -u nginx`: se ven los errores `connect() failed`, `Connection refused`, apuntando al upstream `http://127.0.0.1:8081`, lo que confirma que el 502 se debía a que el backend todavía no estaba levantado en ese puerto cuando hicimos las primeras pruebas.



The screenshot shows a Kali Linux virtual machine environment. On the left, a terminal window displays the output of a curl command to a target URL, showing HTTP headers and a 200 OK status. The main window displays a web application titled 'Inventario de dispositivos' (Inventory of devices) for 'Laboratorio 3, FDSI'. The application shows a table of devices with columns for Name, Type, IP, Vendor, and Status. The table contains five entries: FW-LAB-01 (Firewall, Aruba-Sim, Online), FW-LAB-02 (Firewall, Cisco-Sim, Offline), RTR-LAB-01 (Router, Cisco-Sim, Online), SW-LAB-01 (Switch, Aruba-Sim, Online), and SW-LAB-02 (Switch, Huawei-Sim, Unknown). Each entry has a 'Ver detalle' (View details) link. The right side of the screen shows a 3D rendering of server racks.

Inventario de dispositivos ya poblado con los cinco registros ficticios (FW-LAB-01, FW-LAB-02, RTR-LAB-01, SW-LAB-01 y SW-LAB-02), evidencia de que una vez levantado el backend en el puerto 8081 la API respondió correctamente y la aplicación dejó de mostrar el error 502 que habíamos visto antes.

```
[deltax@deltax ~]$ sudo awk '$9 ~ /404/ {print $1, $4, $7, $9, $12}' /var/log/nginx/access.log | sort |
uniq -c | sort -nr | head
[deltax@deltax ~]$ sudo grep -E 'nmap|curl|ZAP' /var/log/nginx/access.log | tail -n 30
10.212.92.142 - - [09/Sep/2026:09:10:09 -0500] "GET /nmaplowercheck1788963007 HTTP/1.1" 200 350 "-" "Mozilla/5.0 (co
mpatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:10:09 -0500] "POST /sdk HTTP/1.1" 405 157 "-" "Mozilla/5.0 (compatible; Nmap Scrip
ting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:10:09 -0500] "GET /NmapUpperCheck1788963007 HTTP/1.1" 200 350 "-" "Mozilla/5.0 (co
mpatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:10:09 -0500] "GET /Nmap/folder/check1788963007 HTTP/1.1" 200 350 "-" "Mozilla/5.0
(compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:49:25 -0500] "GET /nmaplowercheck1788965363 HTTP/1.1" 200 350 "-" "Mozilla/5.0 (co
mpatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:49:25 -0500] "POST /sdk HTTP/1.1" 405 157 "-" "Mozilla/5.0 (compatible; Nmap Scrip
ting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:49:25 -0500] "GET /NmapUpperCheck1788965363 HTTP/1.1" 200 350 "-" "Mozilla/5.0 (co
mpatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:49:25 -0500] "GET /Nmap/folder/check1788965363 HTTP/1.1" 200 350 "-" "Mozilla/5.0
(compatible; Nmap Scripting Engine; https://nmap.org/book/nse.html)"
10.212.92.142 - - [09/Sep/2026:09:51:23 -0500] "GET / HTTP/1.1" 200 350 "-" "curl/7.88.1"
10.212.92.142 - - [09/Sep/2026:09:52:28 -0500] "GET /public-inventory.txt HTTP/1.1" 200 350 "-" "curl/7.88.1"
[deltax@deltax ~]$
```

Cierre del Paso 13 de la guía, correlación de eventos, en el que intentamos contar los errores 404 con awk sin que ese resultado quedará capturado en la evidencia, y ejecutamos `grep -E 'nmap|curl|ZAP' /var/log/nginx/access.log`. El filtro deja ver rutas como `NmapLowerCheck` y `NmapUpperCheck`, que no son peticiones manuales sino artefactos que genera automáticamente el propio Nmap durante la detección de versión con `-sV`.